REP-003 · DEC-006 confirmed; PLAN-002 (Package B) proposed

Game	Fruit Market
Report date	2026-10-26 13:50 (Asia/Kuala_Lumpur)
Build	`main @ 63c3682` (live game still `c915e49`)
Type	Planning only. **No game implementation changes were made.**
Approval status	DEC-006 **approved** (owner, 2026-10-26: "Confirm DEC-006"). PLAN-002 **awaiting approval**

1 · DEC-006 confirmed

The preview stays a separate staging Worker (`fruit-market-staging`). Approval status: approved; delivery: verified (REP-002, A3).

2 · Problem Package B solves

Records exist, but they're kept by hand:

- Each decision's status is typed in two or three places (the register, every report that mentions it, STATE). Nothing checks that they agree. REP-002 already needed a manual word-count correction.
- Nothing stops an approval being recorded without your words as evidence, or history being edited.
- There's no player-facing changelog and no version tags, so "what changed in this release" can't be answered from the repo.
- The PDF export works (built in REP-001) but isn't part of any check.

Players never see this directly. It matters because it keeps approvals honest and makes releases traceable when there's money or player data involved later.

3 · PLAN-002 · Package B: canonical records, generated views, checks

Included scope

1. **One canonical source per record** (JSON, no new tools):
   - `docs/workflow/records/decisions.json`: every DEC with question, choice, rationale, impacts, owner, approval evidence (your exact words and date), delivery status, links, review trigger, supersession. A `history` list is only ever appended to.
   - `docs/workflow/records/PLAN-###.json`: one per package with scope, exclusions, acceptance, approval evidence, changes (Added / Changed / Fixed / Removed), checks, build, release state.
   - Reports (`reports/REP-###.md`) stay as hand-written narrative snapshots and are never rewritten.
2. **Generated views**, built by `npm run records` with a deterministic script:

    - `DECISIONS.md`: current decisions and pending approvals first, then the full history.
    - `CHANGELOG.md`: player-facing, grouped by release, with "unreleased" vs "published" state. It is seeded with one summary entry for everything before the workflow, taken from git history.
    - The "Pending approvals" block inside `STATE.md` (between markers; the rest stays hand-written).
    - PDFs: each report, plus the cumulative register (current decisions, then pending approvals, then all reports in order).
3. **Checks**, via `npm run records:check`, added to `npm run verify` so CI runs it. It fails when:
    - an ID is duplicated or skipped, or a report mentions a DEC, PLAN or REP that doesn't exist
    - a decision is "approved" without owner evidence, or a delivery status isn't one of the allowed values
    - a history entry was changed or removed compared with the last commit (append-only)
    - a generated view is out of date with its source
4. **Release tagging**: when you publish, the deploy records the release in the changelog and tags the commit (`v1.1.1` and so on), so every live build can be traced. This only runs when you press *Deploy*.
5. **Migration**: move DEC-001 to DEC-006, PLAN-001 and REP-001 to REP-003 into the new records without losing anything.

Excluded

- Game code, rules, server and hosting.
- Package C (devices, backups, capacity).
- A cross-game studio register (NOT APPLICABLE until there's a second game).
- Installing anything. PDF building stays local because it uses pandoc. If pandoc is missing, the script prints PDF EXPORT BLOCKED and keeps the last good PDFs; it isn't part of CI.

Roles: producer (record design), technical lead (scripts), QA (failure cases), DevOps (tagging). Self-review only; low risk.

Likely files: new `docs/workflow/records/*.json`, new `scripts/records.mjs`, new `CHANGELOG.md`, `docs/workflow/export-pdf.cjs` (reads the records), `package.json` (scripts), `.github/workflows/deploy-web.yml` (tag and changelog step), `CLAUDE.md` and `OWNER-BRIEF.md` (one line each).

Acceptance (observable)

- B1. `npm run records` rebuilds all views. Running it twice in a row changes nothing (no git diff in the Markdown outputs).
- B2. `records:check` fails, with a plain-English message, for each of these deliberate breaks (shown, then reverted):
    - a duplicate ID
    - an unknown DEC referenced in a report
    - "approved" without evidence
    - an edited history line
    - a stale `DECISIONS.md`

- B3. With pandoc made unavailable, the export prints PDF EXPORT BLOCKED, exits non-zero and leaves the previous PDFs untouched.
- B4. The cumulative PDF opens with current decisions and pending approvals, followed by REP-001 to REP-003 in order.
- B5. After migration, every decision, status and approval quote from REP-001 to REP-003 appears in the records.
- B6. `npm run verify` (and CI) runs `records:check` and stays green on the migrated records.
- B7. Tagging is tested on a throwaway branch: a dry run shows the tag and changelog entry it would create. The real tag happens only at your next publication.

Checks: B1–B7, plus `verify`, plus one CI run.

Cost and data: RM0. No player data. `npm run verify` gets about 1 second slower.

Risks:

- Record-keeping overhead: JSON is less friendly to edit by hand. Mitigation: Claude edits the records; you only read the generated Markdown and PDFs.
- The append-only check compares with the last commit, so a deliberate correction needs a new "correction" history entry rather than an edit. That is intended.

Rollback: revert the package commits. The hand-written `DECISIONS.md` and reports remain valid.

4 · Decisions

ID	Question	Recommendation	Status	Delivery
DEC-006	Preview = separate staging Worker	Keep	Approved (owner, 2026-10-26)	Verified
DEC-007	Package B scope = PLAN-002	As above	Awaiting approval	Not started
DEC-008	Canonical record format	JSON records plus Markdown narrative reports; Markdown, changelog and PDF views generated	Awaiting approval (part of PLAN-002)	Not started
DEC-009	Records check in `verify` and CI	Yes, so disagreements block merges once branch protection is on	Awaiting approval (part of PLAN-002)	Not started

Options considered for DEC-008:

- Keep hand-written Markdown only: free, but can't be validated reliably.
- YAML front matter: friendlier, but needs a parser dependency.
- A database: ruled out by the workflow guide.
- **JSON (recommended):** Node reads it without dependencies, and the check can be strict.

5 · Still open from earlier (not blocking PLAN-002)

- Which real phones you can test on (needed for Package C).
- Your Cloudflare plan (Free or Workers Paid) and any other paid accounts (DEC-005).
- Preview playtest (REP-002, A4) and branch protection (owner brief, section 5).